Практическая работа 3

Баев Ярослав

1. Ответы на вопросы

Чем отличаются текстовые и бинарные форматы

Текстовые форматы можно открыть и прочитать глазами, но они занимают больше места и работают медленнее. Бинарные форматы занимают мало места и работают быстро, но без специальной программы их не прочитать.

Что такое схемная эволюция

Это когда в данные можно добавить новое поле или убрать старое, и при этом старые программы продолжают работать и не ломаются.

2. Результаты выполнения шагов

Таблица 1. Сравнение текстовых форматов

Формат	Размер (байты)	Читаемость 1-5	Есть ли комментарии
JSON	253	4	Нет
XML	283	3	Да
YAML	182	5	Да

Таблица 2. Сводная таблица всех форматов

Формат	Размер (байты)	Читаем ли человеком	Нужна ли схема
JSON	211	Да	Нет
XML	283	Да	Нет
YAML	182	Да	Нет
MessagePack	157	Нет	Нет
CBOR	159	Нет	Нет
Protobuf	117	Нет	Да

Скриншоты

```
C:\date_format>python run_all.py
Формат          Размер (байт)    Читаем?
-----------------------------------------
JSON                      211         да
XML                       283         да
YAML                      182         да
MessagePack               157        нет
CBOR                      159        нет
Protobuf                  117        нет
C:\date_format>_
```

Скриншот 1. Вывод run_all.py

```
C:\date_format>python step_json.py
=== JSON ===
{
  "id": 42,
  "name": "Анна Смирнова",
  "age": 22,
  "is_student": true,
  "courses": [
    "Математика",
    "Физика",
    "Информатика"
  ],
  "address": {
    "city": "Москва",
    "zip": "101000"
  }
}
Размер: 253 байт
Десериализация успешна: Анна Смирнова
C:\date_format>_
```

Скриншот 2. Вывод step_json.py

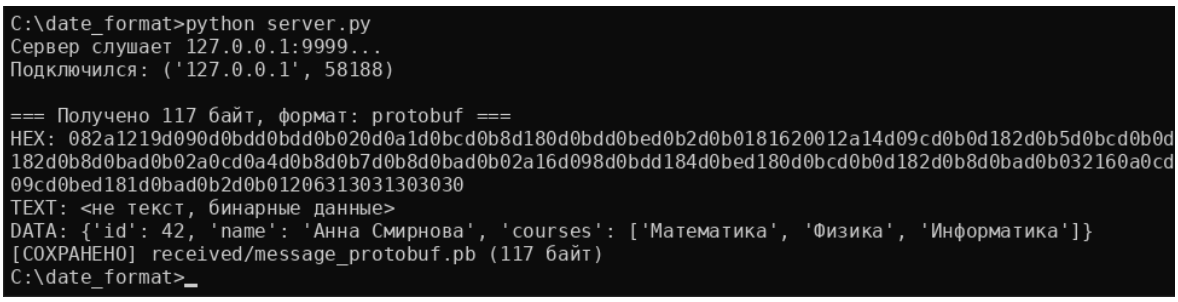
```
C:\date_format>python step_protobuf.py
=== Protobuf ===
HEX: 082a1219d090d0bdd0bdd0b020d0a1d0bcd0b8d180d0bdd0bed0b2d0b0181620012a14d09cd0b0d182d0b5d0bcd0b0d
182d0b8d0bad0b02a0cd0a4d0b8d0b7d0b8d0bad0b02a16d098d0bdd184d0bed180d0bcd0b0d182d0b8d0bad0b032160a0cd
09cd0bed181d0bad0b2d0b01206313031303030
Размер: 117 байт
Десериализация успешна: Анна Смирнова
C:\date_format>_
```

Скриншот 3. Вывод step_protobuf.py

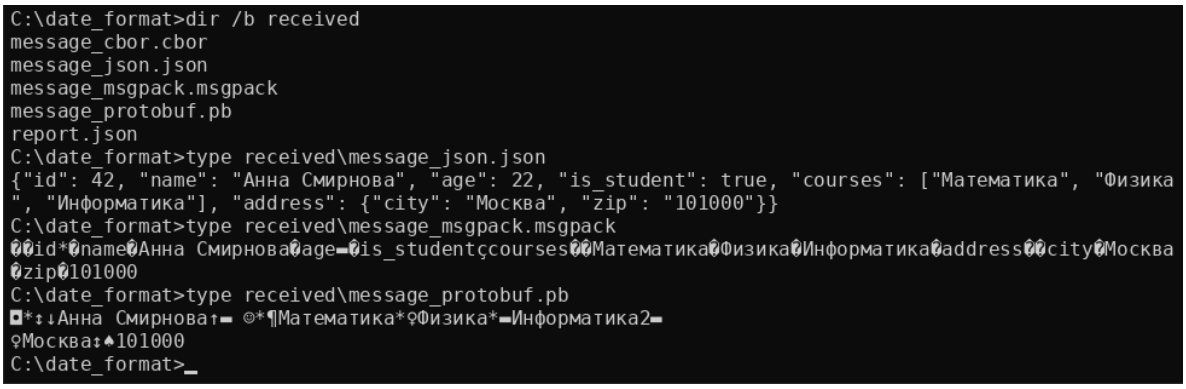
```
C:\date_format>python server.py
Сервер слушает 127.0.0.1:9999...
Подключился: ('127.0.0.1', 58170)

=== Получено 211 байт, формат: json ===
HEX: 7b226964223a2034322c20226e616d65223a2022d090d0bdd0bdd0b020d0a1d0bcd0b8d180d0bdd0bed0b2d0b0222c2
022616765223a2032322c202269735f73747564656e74223a20747275652c2022636f7572736573223a205b22d09cd0b0d18
2d0b5d0bcd0b0d182d0b8d0bad0b0222c2022d0a4d0b8d0b7d0b8d0bad0b0222c2022d098d0bdd184d0bed180d0bcd0b0d18
2d0b8d0bad0b0225d2c2022616472657373223a207b2263697479223a2022d09cd0bed181d0bad0b2d0b0222c20227a697
0223a2022313031303030227d7d
TEXT: {"id": 42, "name": "Анна Смирнова", "age": 22, "is_student": true, "courses": ["Математика", "
Физика", "Информатика"], "address": {"city": "Москва", "zip": "101000"}}
DATA: {'id': 42, 'name': 'Анна Смирнова', 'age': 22, 'is_student': True, 'courses': ['Математика', '
Физика', 'Информатика'], 'address': {'city': 'Москва', 'zip': '101000'}}
[СОХРАНЕНО] received/message_json.json (211 байт)
C:\date_format>_
```

Скриншот 4. Сервер получил данные в формате JSON



Скриншот 5. Сервер получил данные в формате Protobuf



Скриншот 6. Папка received с файлами разных форматов



Скриншот 7. Проверка схемной эволюции

3. Выводы

1. Почему Protobuf и MessagePack оказались меньше JSON?

В JSON каждое название поля записано словами, и ещё есть много скобок, кавычек и пробелов. В MessagePack числа и строки записаны короткими метками, поэтому лишних символов нет. В Protobuf вместо названий полей стоят их номера, поэтому он самый маленький: 117 байт против 211 у JSON.

2. В каком формате данные было легче всего читать вручную?

Легче всего мне было читать YAML. В нём нет лишних скобок и кавычек, каждое поле на своей строке, а вложенность видна по отступам. В XML много повторяющихся тегов, поэтому читать его тяжелее.

3. Почему JSON не поддерживает схемную эволюцию, а Protobuf поддерживает?

В JSON нет схемы и общих правил, поля просто записаны по названиям. Если программа ждёт поле, а его нет, она может выдать ошибку. В